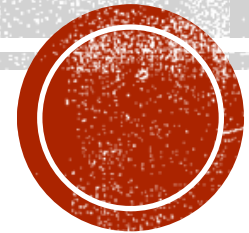




**Faculty of Economics and
Business Administration**

Introduction to Dart programming – PART 2



Dr. Abdul Rahman EL SAYED

OOP IN DART

- Dart has many features that automates and facilitates the process of creating classes and objects. Below are some of Dart's OOP features:
 - Automatic initialization of class fields through class constructors.
 - Implicit getters and setters for all class fields
 - Implicitly calling getters/setters when you access class fields
 - Allows the creation of multiple named constructors



CREATE A SIMPLE CLASS

- We will create a simple class to represent a Circle. We will first show the code using Java and how we can convert it to Dart syntax. Below is the Java code for the Circle class.
- ```
public class Circle {
 double radius;
 // declare a constructor
 public Circle(double r) {
 radius = r;
 }
 // declare a method to return the area of the circle
 public double getArea() {
 return radius * radius * Math.PI;
 }
}
```



# CREATE A SIMPLE CLASS

- Now we will write the same class using Dart. As you can see, the syntax is much simpler than Java. Dart will implicitly create a getter and setter for the radius field.

```
▪ class Circle {
▪ double radius;
▪ // declare the constructor, it will set the radius field.
▪ Circle(this.radius);
▪ // a method to return the area of a circle
▪ double getArea() {
▪ return pi * radius * radius;
▪ }
▪ }
▪
```



# CREATING CLASS OBJECTS

- Below code demonstrate how you can create an object from class circle. You don't need to write the new keyword to create a new object, as is the case for Java.

```
▪ void main() {
▪ // create a new Circle object with radius 10
▪ Circle c1 = Circle(10);
▪ // print its radius and area
▪ print('Radius: ${c1.radius}');
▪ print('Area: ${c1.getArea()}');
▪ // change the radius to 20
▪ // the below code will call the setter for the field radius.
▪ // in java you write: setRadius(20);
▪ c1.radius = 20;
▪ }
```



# PRIVATE CLASS FIELDS

- You create private class fields by prefixing the field name by “\_”. This makes the field private inside the file(library) it is defined in. Private fields in Dart are not private inside the class but inside the file. You need to put the Circle class inside its own file.

```
class Circle {
 double _radius = 0; // make radius private

 Circle(double r) {
 _radius = r; // call the radius setter
 }

 double getArea() {
 return pi * _radius * _radius;
 }
}
```



# ADDING SETTERS AND GETTERS

- You can add a setter and getter for the radius as shown below:

```
▪ // define a getter for radius
▪ double get radius => _radius;
▪ // add a setter to validate the radius value
▪ set radius(double r) {
▪ if (r > 0) {
▪ _radius = r;
▪ } else {
▪ throw Exception('Invalid Radius');
▪ }
▪ }
▪
▪
```



# ACCESSING PRIVATE FIELDS

- You can add access private class members using getters and setters.

```
import 'circle.dart'; // import the library of the circle class

void main() {
 try {
 // the constructor will call the radius setter
 Circle c1 = Circle(10);
 print('Radius: ${c1.radius}'); // call the radius getter
 // call the radius setter to change it to 20
 c1.radius = -2;
 // The above will generate an exception, since the radius is negative
 } catch (e) {
 print('$e');
 }
}
```



# ADDING A toString METHOD

- You can add a toString method to print information about the class, just the same way you use it in Java. If a class does not override the toString method, it inherits it from the Object class. All classes in Dart inherits from the Object class.
- `// toString returns information about the class`
- `@override`
- `String toString() {`
- `return 'Radius: $_radius Area: ${getArea()}';`
- `}`
- You can then call it as shown below:
- `print('$c1'); // call the toString method`



# NAMED CONSTRUCTORS

- Unlike Java and many other languages, Dart does not have overloading by redefinition, so, to define alternative constructors for a class, you need to use the named constructors. A named constructor is how you define alternative constructors for a class. Below we define a named constructor called create, to create a new circle with radius 5.

- `// add a named constructor to initialize the radius to 5`

- `Circle.create() : _radius = 5;`

▪

You can then create a new circle using the named constructor, as shown below:

- `Circle c2 = Circle.create();`

▪

▪

▪



# CLASS INHERITANCE

- In addition to the implicit inheritance to the Object type, Dart allows us to extend defined classes by using the extends keyword, where all of the members of the parent class are inherited, except the constructors. We will first create a super class called Person.

```
▪ class Shape {
▪ String color;
▪
▪ Shape(this.color);
▪
▪ @override
▪ String toString() {
▪ return 'Color: $color';
▪ }
▪ }
```



# CLASS INHERITANCE

- Now we will create a simplified version of the Circle class that extends from the Shape class.

```
▪ class Circle extends Shape {
▪ double radius;
▪
▪ Circle(String color, this.radius)
▪ : super(color); // call parent constructor to set color
▪
▪ @override
▪ String toString() {
▪ return '${super.toString()} Radius: $radius';
▪ }
▪ }
```



# CLASS INHERITANCE

- The below code demonstrates the use of the Shape and Circle classes.

- `void main() {`
- `// create a shape object`
- `Shape s = Shape('green');`
- `print('Shape: $s');`
- `// create a circle object`
- `Circle c = Circle('red', 10);`
- `print('Circle: $c');`
- `}`

